

OPEN IT SUMMER

BOOTCAMP

DAY 1A

Data Formats, REST APIs & HATEOAS

JSON • XML • Schemas • HTTP • REST Levels 0–3 • Resource Routing

Duration: ~2 hours | Mini activities + 2-hour hands-on lab

DATA FORMATS & REST

Session Overview

01

Data Serialization — JSON & XML

02

HTTP Request & Response Headers

03

REST Levels 0 → 3 (Richardson Maturity Model)

04

HTTP Verbs, Status Codes & Resources

05

Resource-Based Routing — `/books/1/authors/2`

06

Query Params vs Path Params — `/books/1` vs `/books?id=1`

07

HATEOAS — Level 3 REST in Depth

08

2-Hour Hands-On Lab

What is Data Serialization?

Converting objects → transmittable bytes, and back

Why It Matters

- Send data over HTTP APIs
- Store config files on disk
- Inter-process communication
- Cross-language data exchange
- Database I/O

Common Formats

- JSON — lightweight, web-native
- XML — verbose, schema-driven
- YAML — human-friendly configs
- Protobuf — binary, very fast
- MessagePack — compact binary

Key Concepts

- Serialize → encode to bytes/text
- Deserialize → decode back to object
- Schema → structural contract
- Validation → enforce the schema
- Versioning → backward compat

JSON — JavaScript Object Notation

The de-facto standard for REST APIs (ECMA-404)

String

"Hello World"

Number

42, 3.14, -7

Boolean

true / false

Null

null

Array

[1, "a", true]

Object

{ "key": value }

Example JSON

```
{
  "student": {
    "id": 101,
    "name":
      "Maria Santos",
    "enrolled": true,
    "grades":
      [88, 92, 95],
    "address": null
  }
}
```

JSON Schema — Validate Your Data

Enforce structure and data types as a contract

Schema Definition

```
{
  "$schema":
    "https://json-schema.org/draft-07/schema",
  "type": "object",
  "required": ["id", "name"],
  "properties": {
    "id": {
      "type": "integer"
    },
    "name": {
      "type": "string",
      "minLength": 1,
      "maxLength": 100
    },
    "age": {
      "type": "integer",
      "minimum": 0,
      "maximum": 120
    },
    "email": {
      "type": "string",
      "pattern": "^[^@]+@[^@]+$"
    }
  }
}
```

Schema Keywords

type	— Data type constraint
required	— Mandatory fields list
properties	— Field definitions
minLength / maxLength	— String length bounds
minimum / maximum	— Numeric bounds
pattern	— Regex validation
enum	— Allowed value list
\$ref	— Sub-schema reference

JSON Modeling — Model a Library Book

Create a JSON object for a "Library Book" with these fields:

- id (integer)
- available (boolean)
- title (string)
- genres (array of strings, 2+)
- authors (array of strings)
- publisher { name, city }
- published_year (integer)

Then write a JSON Schema that validates your object.

Require at least 3 fields with type constraints.

Tool: Test your schema at jsonschemavalidator.net

XML — eXtensible Markup Language

Structured, schema-driven, enterprise-proven

XML Document Example

```
<?xml version="1.0" encoding="UTF-8"?>
<library xmlns="http://example.com/lib">
  <book id="101" genre="tech">
    <title>Clean Code</title>
    <author>Robert C. Martin</author>
    <year>2008</year>
    <price currency="USD">35.99</price>
    <available>true</available>
    <tags>
      <tag>programming</tag>
      <tag>best-practices</tag>
    </tags>
  </book>
</library>
```

Core Concepts

Elements	<tag>content</tag>
Attributes	id="101" in opening tag
Root Element	One top-level element only
Namespace	xmlns="..." avoids conflicts
CDATA	<![CDATA[raw text]]>
Declaration	<?xml version="1.0"?>
Comments	<!-- comment here -->
Encoding	UTF-8, UTF-16, etc.

XML Schema (XSD)

Define and enforce XML document structure

XSD Definition

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <xs:element name="book">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="title" type="xs:string" minOccurs="1"/>
        <xs:element name="author" type="xs:string" minOccurs="1" maxOccurs="unbounded"/>
        <xs:element name="year" type="xs:integer" minOccurs="0"/>
      </xs:sequence>
      <xs:attribute name="id" type="xs:integer" use="required"/>
    </xs:complexType>
  </xs:element>
</xs:schema>
```

xs:element	— Defines an XML element with name and type
xs:complexType	— Element that contains child elements or attributes
xs:sequence	— Children must appear in the defined order
minOccurs / maxOccurs	— Cardinality — "unbounded" allows any count
xs:attribute	— Defines an attribute on the parent element (use="required")

XML Structuring — Convert Your JSON Book to XML

Using the Library Book JSON you modeled in the last activity:

1. Convert it to an XML document with proper `<?xml?>` declaration and UTF-8 encoding
2. Include at least one attribute (e.g., `id="101"` on the `<book>` element)
3. Add a namespace declaration to the root element
4. Model the authors array as repeated `<author>` elements
5. Write a minimal XSD snippet that validates: title (required), id (attribute, integer), year (optional integer)

Bonus: Add a `<!-- comment -->` describing the document's purpose

Tool: Validate at [xmlvalidation.com](https://www.xmlvalidation.com)

JSON vs XML — Side by Side

Choose the right format for the job

Feature	JSON	XML
Verbosity	Compact, minimal syntax	Verbose, tag-heavy
Readability	Very easy for devs	Readable but wordy
Schema	JSON Schema (optional)	XSD / DTD (strict)
Data Types	String, Number, Bool, Null, Array, Object	Everything is text
Comments	Not natively supported	<!-- supported -->
Browser	Native JSON.parse()	DOMParser / XSLT
Best For	REST APIs, web, configs	Enterprise, SOAP, B2B

REST API — Core Principles

Representational State Transfer (Roy Fielding, 2000)

Stateless

Each request contains all info needed. No server-side session.

Uniform Interface

Standard HTTP verbs, resources identified by URIs.

Layered System

Client cannot tell if connected to end server or proxy.

Client–Server

UI and data are decoupled. Clients call resources via URIs.

Cacheable

Responses declare cacheability to improve scalability.

HATEOAS

Hypermedia links guide the client to next actions (Level 3).

HTTP Verbs & Resource Design

The building blocks of REST APIs

GET

Read — Retrieve resource or collection

```
GET /books
GET /books/101
```

POST

Create — Submit a new resource

```
POST /books
```

PUT

Replace — Full update of a resource

```
PUT /books/101
```

PATCH

Update — Partial update of a resource

```
PATCH /books/101
```

DELETE

Remove — Delete a resource

```
DELETE /books/101
```

HTTP Request — Anatomy

Every API call is an HTTP message with a clear structure

Full HTTP Request

```
POST /api/books HTTP/1.1
Host: api.library.com
Content-Type: application/json
Accept: application/json
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
Accept-Language: en-US
Cache-Control: no-cache
X-Request-ID: abc-123-def

{ "title": "Clean Code", "author": "Robert Martin", "year": 2008 }
```

Request Line

— Method + Path + HTTP version — defines what action to perform and where

Headers

— Key-value metadata — content type, auth token, accepted formats, caching rules

Blank Line

— Mandatory separator between headers and body (CRLF)

Body

— Payload — JSON, XML, or form data. Only present on POST/PUT/PATCH requests

HTTP Request Headers — Deep Dive

Metadata that tells the server how to handle the request

Content-Type

`application/json`

Format of the request body being sent to the server

Accept

`application/json, application/xml`

Formats the client is willing to receive in response

Authorization

`Bearer <JWT> or Basic <base64>`

Authentication credentials — JWT, API Key, Basic Auth

Host

`api.library.com`

Target domain — required in HTTP/1.1 for virtual hosting

User-Agent

`Mozilla/5.0 / PostmanRuntime/7.x`

Client app identifier — browser, Postman, mobile app

Cache-Control

`no-cache / max-age=3600`

Caching directives — how long the response can be stored

Accept-Language

`en-US, tl`

Preferred response language for content negotiation

Accept-Encoding

`gzip, deflate, br`

Accepted compression algorithms to reduce payload size

If-None-Match

`"abc123etag"`

ETag from a previous response — used for conditional GETs

X-Request-ID

`uuid-abc-123`

Custom header — tracing & correlation across microservices

HTTP Response — Headers & Structure

What the server sends back after processing your request

Full HTTP Response

```
HTTP/1.1 201 Created
Content-Type: application/json; charset=utf-8
Location: /api/books/42
ETag: "abc123def456"
Cache-Control: no-store
X-RateLimit-Remaining: 98
Date: Sat, 03 May 2026 08:00:00 GMT

{ "id": 42, "title": "Clean Code", "available": true }
```

Content-Type	application/json; charset=utf-8	→ Format and encoding of the response body
Location	/api/books/42	→ 201 Created → URL of the newly created resource
ETag	"abc123def456"	→ Entity tag — checksum for cache validation (If-None-Match)
Cache-Control	no-store / max-age=3600	→ Tells client how long to cache the response
X-RateLimit-Remaining	98	→ Custom: API rate limit — how many calls left this window
WWW-Authenticate	Bearer realm="api"	→ On 401 — tells client what auth scheme is required
Access-Control-Allow-Origin	* or https://app.com	→ CORS header — which origins may access this resource

Richardson Maturity Model — REST Levels 0 to 3

A scale for measuring how "RESTful" an API truly is

Defined by Leonard Richardson (2008) and popularized by Martin Fowler. Each level adds constraints that increase API quality.

Level 0

The Swamp of POX

Plain Old XML / one endpoint, one action

- Single URL for everything
- Action defined in the body
- HTTP is just a tunnel
- No use of HTTP semantics

Level 1

Resources

Individual resources get their own URIs

- Separate URL per resource
- /books, /authors, /loans
- Still only POST typically
- Better structure, poor verbs

Level 2

HTTP Verbs

Use GET, POST, PUT, DELETE correctly

- Right verb for right action
- GET is safe, idempotent
- Correct status codes returned
- Most public APIs stop here

Level 3

Hypermedia (HATEOAS)

Responses carry links to next actions

- _links in every response
- Self-describing API
- Client needs no hardcoded URLs
- True RESTful architecture

Level 0 & Level 1 — Explained with Code

From one-endpoint-does-all to resource-per-URL

Level 0 — Single Endpoint (RPC style)

```
// ALL calls go to ONE URL
// Action is buried in the body

// Get a book
POST /api
{ "action": "getBook", "id": 1 }

// Create a book
POST /api
{ "action": "createBook",
  "title": "Clean Code" }

// Delete a book
POST /api
{ "action": "deleteBook", "id": 1 }

// Problems:
// X No HTTP semantics used
// X Can't cache GET responses
// X Not self-describing
// X SOAP and XML-RPC live here
```

Level 1 — Resources (separate URLs)

```
// Each resource has its own URL
// But still uses POST for everything

// Get a book (should be GET, but...)
POST /api/books
{ "action": "get", "id": 1 }

// Get all books
POST /api/books
{ "action": "list" }

// Create a book
POST /api/books
{ "action": "create",
  "title": "Clean Code" }

// Progress:
// ✓ Separate URL per resource
// X Still misuses HTTP verbs
// X No standard status codes
// X Caching is broken
```

Key insight: Level 0 and Level 1 APIs work — but they ignore the power HTTP gives you for free (caching, semantics, proxies).

Level 2 — HTTP Verbs & Status Codes

Use the right verb for the right action. This is where most APIs live.

Level 2 in Practice

GET

/api/books

→ 200 OK + [array of books]

GET

/api/books/1

→ 200 OK + { book object } | 404 Not Found

POST

/api/books

→ 201 Created + { new book } | 400 Bad Request

PUT

/api/books/1

→ 200 OK + { updated book } | 404 Not Found

PATCH

/api/books/1

→ 200 OK + { patched book }

DELETE

/api/books/1

→ 204 No Content | 404 Not Found

GET	Safe + Idempotent	Retrieve only. Never changes server state. Can be cached by browsers and proxies.
POST	Neither	Creates a NEW resource. Not idempotent — calling twice creates two records.
PUT	Idempotent	Full replacement of a resource. Same call twice has same result.
PATCH	Neither usually	Partial update. Only send the fields you want changed.
DELETE	Idempotent	Removes resource. Second call returns 404 — same outcome (resource gone).

HTTP Status Codes — Complete Reference

2xx — Success

200 OK	— GET succeeded, or PUT/PATCH updated
201 Created	— POST succeeded — resource created
204 No Content	— DELETE succeeded — no body returned

3xx — Redirect

301 Moved Permanently	— Resource has a permanent new URL
304 Not Modified	— ETag matched — use your cached copy

4xx — Client Error

400 Bad Request	— Malformed JSON / validation failed
401 Unauthorized	— No or invalid auth token provided
403 Forbidden	— Authenticated but lacks permission
404 Not Found	— Resource does not exist at that URI
405 Method Not Allowed	— HTTP verb not supported on this URL
409 Conflict	— Duplicate resource / state conflict
422 Unprocessable Entity	— Semantically invalid request body

5xx — Server Error

500 Internal Server Error	— Unhandled exception on the server
503 Service Unavailable	— Server down or overloaded

/books/1 vs /books?id=1 — What's the Difference?

Path parameters vs query parameters — when to use which

Path Parameter — /books/1	Query Parameter — /books?id=1
<pre>// "1" is part of the URL path itself // The resource IS identified by this value GET /api/books/1 GET /api/authors/42 GET /api/users/99/orders/7</pre>	<pre>// id=1 is appended after the "?" // Used for filtering, searching, sorting GET /api/books?id=1 GET /api/books?genre=fiction&available=true GET /api/books?search=clean&sort=year&page=2</pre>

Aspect	/books/1 (path)	/books?id=1 (query)
Purpose	Identify a SPECIFIC resource	Filter / search / sort / paginate a collection
RESTful use?	✓ Preferred for single resource identity	✓ Fine for optional modifiers on collections
Required?	Yes — missing it changes the route entirely	No — optional, server sets defaults
Cacheability	Excellent — URL is unique per resource	Varies — caches treat each combo as different
Example	/books/1 /users/42 /orders/7	/books?genre=fiction&sort=title&page=2
Looks RESTful?	Yes — clean, hierarchical	Less clean — but OK for collection filters

Query Parameters — In Depth

Filtering, searching, sorting, and pagination

Query Parameter Anatomy

```
https://api.library.com/books?genre=fiction&available=true&sort=year&order=desc&page=2&size=10
```

└─ key=value pairs separated by & ─┐

? marks the start of the query string

Filtering

?genre=fiction

?available=true

?year=2020

?author=Martin

Searching

?search=clean+code

?q=robert+martin

?title=pragmatic

Sorting

?sort=title&order=asc

?sort=year&order=desc

?sortBy=author

Pagination

?page=2&size=10

?offset=20&limit=10

?cursor=abc123

Rule of thumb: Path params identify a resource. Query params modify how you retrieve a collection.

Resource-Based Routing

REST URLs should read like nouns, not verbs. Resources nest to show ownership and relationships.
Hierarchical URLs that mirror real-world relationships

Resource URL Patterns

GET	/books	→ All books (collection)
GET	/books/1	→ Book with id=1 (single resource)
GET	/books/1/authors	→ All authors of book 1
GET	/books/1/authors/2	→ Author 2 of book 1 (nested resource)
GET	/books/1/reviews	→ All reviews for book 1
GET	/books/1/reviews/5	→ Review 5 on book 1
POST	/books/1/reviews	→ Add a review to book 1
GET	/authors/2/books	→ All books written by author 2
GET	/libraries/3/books?available=true	→ Available books in library 3

✓ Use NOUNS for resources	✓ /books, /authors, /loans	X /getBooks, /createAuthor, /deleteBook
---------------------------	----------------------------	---

✓ Use PLURAL for collections	✓ /books, /users, /orders	X /book, /user, /order
------------------------------	---------------------------	------------------------

✓ Nest to show relationships	✓ /books/1/authors	X /getAuthorsOfBook?bookId=1
------------------------------	--------------------	------------------------------

⚠ Keep nesting shallow	✓ /books/1/reviews (2 levels)	X /books/1/chapters/3/paragraphs/7/words (too deep!)
------------------------	-------------------------------	--

Deep Dive: /books/1/authors/2

Reading a nested resource URL step by step

/ books / 1 / authors / 2

/books

Collection of all books

/1

Specific book with id=1

/authors

Sub-collection: authors of that book

/2

Specific author with id=2

All valid routes for this structure

```
GET /books          → all books
GET /books/1        → book 1
GET /books/1/authors → all authors of book 1
GET /books/1/authors/2 → author 2 of book 1
POST /books/1/authors → add an author to book 1
PUT /books/1/authors/2 → update author 2 of book 1
DELETE /books/1/authors/2 → remove author from book 1
```

Response for GET /books/1/authors/2

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "id": 2,
  "fullName": "Robert C. Martin",
  "nationality": "American",
  "_links": {
    "self": { "href": "/books/1/authors/2" },
    "book": { "href": "/books/1" },
    "allBooks": { "href": "/authors/2/books" }
  }
}
```

Level 3 — HATEOAS in Depth

The client starts at a known entry point and discovers everything else from links in responses. No hardcoded URLs needed.
Hypermedia As The Engine Of Application State

HATEOAS Response — GET /books/1

```
HTTP/1.1 200 OK
Content-Type: application/hal+json

{
  "id": 1,
  "title": "Clean Code",
  "available": true,
  "genre": "Programming",
  "_links": {
    "self": { "href": "/books/1",
              "method": "GET" },
    "update": { "href": "/books/1",
                "method": "PUT" },
    "delete": { "href": "/books/1",
                "method": "DELETE" },
    "borrow": { "href": "/books/1/borrow",
                "method": "POST" },
    "reviews": { "href": "/books/1/reviews",
                 "method": "GET" },
    "authors": { "href": "/books/1/authors",
                 "method": "GET" },
    "collection": { "href": "/books",
                    "method": "GET" }
  }
}
```

Why HATEOAS Matters

self link

Always present — canonical URL of the resource itself

Conditional links

Only show "borrow" if available === true — driven by state

Method hints

Tell client which HTTP verb to use for each action

HAL / Siren / JSON:API

Popular hypermedia formats that standardize _links structure

No hardcoding needed

Client never builds /books/1 — it FOLLOWS the link from /books

Self-documenting

The API tells you what you can do next at every step

HATEOAS — Client Navigation Flow

A HATEOAS client just follows links — like browsing the web

1

Entry Point

→ GET /api

← { "_links": { "books": "/books", "authors": "/authors" } }

2

Browse Books

→ GET /books (from link)

← { "items": [...], "_links": { "next": "/books?page=2" } }

3

Select Book

→ GET /books/1 (from item link)

← { ..., "_links": { "borrow": "/books/1/borrow" } }

4

Take Action

→ POST /books/1/borrow (from _links.borrow)

← { "loanId": 99, "dueDate": "2026-06-01" }

Design a Movie Watchlist REST API — Levels 2 & 3

Scenario: Build a REST API for a Movie Watchlist (title, year, genre, watched: bool, rating: float)

1. Design 6+ endpoints using correct HTTP verbs (Level 2):

GET /movies GET /movies/{id} POST /movies
PUT /movies/{id} DELETE /movies/{id} GET /movies/{id}/reviews

2. For GET /movies — add query parameter support:

?genre=action & ?watched=false & ?sort=rating&order=desc & ?page=1&size=5

3. Explain the difference between GET /movies/5 and GET /movies?id=5

When should you use each? (discuss with a classmate)

4. Add a HATEOAS _links section to GET /movies/{id} with at least 5 links

5. What HTTP status codes would you return for:

Successful creation? Resource not found? Validation error?

Group discussion: Compare your URL structures with a classmate!

Library Management System — REST API Design to HATEOAS

1

Part 1 — JSON & XML (30 min)

Create books.json with 5+ books. Write a JSON Schema. Convert to books.xml with XSD. Parse and filter by genre.

2

Part 2 — REST API Design (30 min)

Design /api/books with 8 endpoints. Include /books/{id}/authors/{aid}. Define all query params for filtering and pagination.

3

Part 3 — HTTP Headers & Status Codes (30 min)

Document request headers (Content-Type, Authorization, Accept) and correct status codes for every endpoint.

4

Part 4 — HATEOAS (30 min)

Add _links to all responses. Implement the navigation flow: GET /api → GET /books → GET /books/1 → POST /books/1/borrow.

References & Further Reading

Day 1A

[1] JSON Specification

ECMA-404 Standard · ecma-international.org

[2] JSON Schema Draft 2020-12

json-schema.org · json-schema.org

[3] W3C XML 1.0 Fifth Edition

[w3.org/TR/xml/](https://www.w3.org/TR/xml/) · [w3.org](https://www.w3.org)

[4] REST Architectural Style

Fielding Dissertation, 2000 · ics.uci.edu/~fielding

[5] Richardson Maturity Model

Martin Fowler Blog · martinfowler.com/articles/richardsonMaturityModel.html

[6] HAL Hypermedia Format

Mike Kelly — [application/hal+json](https://stateless.co/hal_specification.html) · stateless.co/hal_specification.html

[7] HTTP Headers Reference

MDN Web Docs · developer.mozilla.org/HTTP/Headers

[8] HTTP Status Codes

MDN Web Docs · developer.mozilla.org/HTTP/Status